Statistical Computing HW4

Name: 莊明儒 Student ID: 111024513

1 Problem 1

1.1 Theories

Let $\boldsymbol{\alpha} = (\alpha_0, \alpha_1)'$ and $\boldsymbol{\beta} = (\beta_0, \beta_1)'$. The likelihood function is

$$\mathcal{L}(\boldsymbol{\alpha}, \boldsymbol{\beta}) \propto \prod_{i=1}^{n} \frac{1}{\sigma_i} \exp\left\{-\frac{1}{2\sigma_i^2}(y_i - \mu_i)^2\right\},$$

where $\mu_i = \beta_0 + \beta_1 x_i$ and $\sigma_i^2 = e^{\alpha_0 + \alpha_1 x_i}$.

We define the MLE for $\hat{\boldsymbol{\alpha}}$ and $\hat{\boldsymbol{\beta}}$ as follows.

$$\begin{aligned}
(\hat{\boldsymbol{\alpha}}, \hat{\boldsymbol{\beta}}) &= \arg \max_{(\boldsymbol{\alpha}, \boldsymbol{\beta}) \in R^2 \times R^2} \mathcal{L}(\boldsymbol{\alpha}, \boldsymbol{\beta}) \\
&= \arg \max_{(\boldsymbol{\alpha}, \boldsymbol{\beta}) \in R^2 \times R^2} l(\boldsymbol{\alpha}, \boldsymbol{\beta}) \\
&= \arg \min_{(\boldsymbol{\alpha}, \boldsymbol{\beta}) \in R^2 \times R^2} -l(\boldsymbol{\alpha}, \boldsymbol{\beta}),
\end{aligned}$$

where

$$\begin{aligned}
l(\boldsymbol{\alpha}, \boldsymbol{\beta}) &= \sum_{i=1}^{n} -\frac{1}{2}\log(\sigma_i) - \frac{1}{2\sigma_i^2}(y_i - \mu_i)^2 \\
&= \sum_{i=1}^{n} -\frac{1}{2}(\alpha_0 + \alpha_1 x_i) - \frac{1}{2e^{\alpha_0 + \alpha_1 x_i}}(y_i - \beta_0 - \beta_1 x_i)^2.
\end{aligned}$$

We have

$$\begin{aligned}
\frac{\partial}{\partial \boldsymbol{\alpha}} l(\boldsymbol{\alpha}, \boldsymbol{\beta}) &= \frac{\partial}{\partial \boldsymbol{\alpha}} \sum_{i=1}^{n} -\frac{1}{2}(\alpha_0 + \alpha_1)x_i - \frac{1}{2e^{\alpha_0 + \alpha_1 x_i}}(y_i - \beta_0 - \beta_1 x_i)^2 \\
&= \left(-\frac{n}{2} + \sum_{i=1}^{n}\left\{\frac{1}{2}e^{-(\alpha_0 + \alpha_1 x_i)}(y_i - \beta_0 - \beta_1 x_i)^2\right\}, \sum_{i=1}^{n}\left\{-\frac{x_i}{2} + \frac{1}{2}x_i e^{-(\alpha_0 + \alpha_1 x_i)}(y_i - \beta_0 - \beta_1 x_i)^2\right\}\right)',
\end{aligned}$$

$$\begin{aligned}
\frac{\partial}{\partial \boldsymbol{\alpha} \partial \boldsymbol{\alpha}^T} l(\boldsymbol{\alpha}, \boldsymbol{\beta}) &= \frac{\partial}{\partial \boldsymbol{\alpha}} \left(-\frac{n}{2} + \sum_{i=1}^{n}\left\{\frac{1}{2}e^{-(\alpha_0 + \alpha_1 x_i)}(y_i - \beta_0 - \beta_1 x_i)^2\right\}, \sum_{i=1}^{n}\left\{-\frac{x_i}{2} + \frac{1}{2}x_i e^{-(\alpha_0 + \alpha_1 x_i)}(y_i - \beta_0 - \beta_1 x_i)^2\right\}\right)' \\
&= \begin{pmatrix} \sum_{i=1}^{n}\left\{-\frac{1}{2}e^{-(\alpha_0 + \alpha_1 x_i)}(y_i - \beta_0 - \beta_1 x_i)^2\right\} & \sum_{i=1}^{n}\left\{-\frac{1}{2}x_i e^{-(\alpha_0 + \alpha_1 x_i)}(y_i - \beta_0 - \beta_1 x_i)^2\right\} \\ \sum_{i=1}^{n}\left\{-\frac{1}{2}x_i e^{-(\alpha_0 + \alpha_1 x_i)}(y_i - \beta_0 - \beta_1 x_i)^2\right\} & \sum_{i=1}^{n}\left\{-\frac{1}{2}x_i^2 e^{-(\alpha_0 + \alpha_1 x_i)}(y_i - \beta_0 - \beta_1 x_i)^2\right\} \end{pmatrix},
\end{aligned}$$

$$
\begin{aligned}
\frac{\partial}{\partial \boldsymbol{\beta}} l(\boldsymbol{\alpha}, \boldsymbol{\beta}) &= \frac{\partial}{\partial \boldsymbol{\beta}} \sum_{i=1}^{n} -\frac{1}{2}(\alpha_0 + \alpha_1 x_i) - \frac{1}{2e^{\alpha_0+\alpha_1 x_i}}(y_i - \beta_0 - \beta_1 x_i)^2 \\
&= \left(\sum_{i=1}^{n} \left\{ e^{-(\alpha_0+\alpha_1 x_i)}(y_i - \beta_0 - \beta_1 x_i) \right\}, \sum_{i=1}^{n} \left\{ x_i e^{-(\alpha_0+\alpha_1 x_i)}(y_i - \beta_0 - \beta_1 x_i) \right\} \right)',
\end{aligned}
$$

and

$$
\begin{aligned}
\frac{\partial}{\partial \boldsymbol{\beta} \partial \boldsymbol{\beta}^T} l(\boldsymbol{\alpha}, \boldsymbol{\beta}) &= \frac{\partial}{\partial \boldsymbol{\beta}} \left(\sum_{i=1}^{n} \left\{ e^{-(\alpha_0+\alpha_1 x_i)}(y_i - \beta_0 - \beta_1 x_i) \right\}, \sum_{i=1}^{n} \left\{ x_i e^{-(\alpha_0+\alpha_1 x_i)}(y_i - \beta_0 - \beta_1 x_i) \right\} \right)' \\
&= \begin{pmatrix} \sum_{i=1}^{n} \left\{ -e^{-(\alpha_0+\alpha_1 x_i)} \right\} & \sum_{i=1}^{n} \left\{ -x_i e^{-(\alpha_0+\alpha_1 x_i)} \right\} \\ \sum_{i=1}^{n} \left\{ -x_i e^{-(\alpha_0+\alpha_1 x_i)} \right\} & \sum_{i=1}^{n} \left\{ -x_i^2 e^{-(\alpha_0+\alpha_1 x_i)} \right\} \end{pmatrix}
\end{aligned}
$$

Recall that our objective is to find $\arg\min_{(\boldsymbol{\alpha},\boldsymbol{\beta}) \in R^2 \times R^2} -l(\boldsymbol{\alpha}, \boldsymbol{\beta})$. Having these results, we can write the algorithm for coordinate descent in the following.

1.2 Algorithm

Step1: For $t = 0$, initialize $\boldsymbol{\alpha}^{(t)} = (0, 0)'$ and $\boldsymbol{\beta}^{(t)} = (0, 0)'$.

Step2: (Set $\boldsymbol{\alpha}^{(t+1)}$ by $\arg\min_{\boldsymbol{\alpha} \in R^2} -l(\boldsymbol{\alpha}, \boldsymbol{\beta}^{(t)})$ using Newton-Rhaphson method.)

- **Step2a**: For $t_{\boldsymbol{\alpha}} = 0$, initialize $\boldsymbol{\alpha}^{(t_{\boldsymbol{\alpha}})} = (0, 0)'$.
- **Step2b**: $\boldsymbol{\alpha}^{(t_{\boldsymbol{\alpha}}+1)} \leftarrow \boldsymbol{\alpha}^{(t_{\boldsymbol{\alpha}})} - \left\{ \frac{\partial}{\partial \boldsymbol{\alpha} \partial \boldsymbol{\alpha}^T} l(\boldsymbol{\alpha}^{(t_{\boldsymbol{\alpha}})}, \boldsymbol{\beta}^{(t)}) \right\}^{-1} \frac{\partial}{\partial \boldsymbol{\alpha}} l(\boldsymbol{\alpha}^{(t_{\boldsymbol{\alpha}})}, \boldsymbol{\beta}^{(t)})$
- **Step2c**: $t_{\boldsymbol{\alpha}} \leftarrow t_{\boldsymbol{\alpha}} + 1$
- **Step2d**: Do **Step2b** and **Step2c** iteratively until $\|\boldsymbol{\alpha}^{(t_{\boldsymbol{\alpha}})} - \boldsymbol{\alpha}^{(t_{\boldsymbol{\alpha}}-1)}\|_2 < 0.001$.
- **Step2e**: $\boldsymbol{\alpha}^{(t+1)} \leftarrow \boldsymbol{\alpha}^{(t_{\boldsymbol{\alpha}}+1)}$

Step3: (Set $\boldsymbol{\beta}^{(t+1)}$ by $\arg\min_{\boldsymbol{\beta} \in R^2} -l(\boldsymbol{\alpha}^{(t+1)}, \boldsymbol{\beta})$ using Newton-Rhaphson method.)

- **Step3a**: For $t_{\boldsymbol{\beta}} = 0$, initialize $\boldsymbol{\beta}^{(t_{\boldsymbol{\beta}})} = (0, 0)'$.
- **Step3b**: $\boldsymbol{\beta}^{(t_{\boldsymbol{\beta}}+1)} \leftarrow \boldsymbol{\beta}^{(t_{\boldsymbol{\beta}})} - \left\{ \frac{\partial}{\partial \boldsymbol{\beta} \partial \boldsymbol{\beta}^T} l(\boldsymbol{\alpha}^{(t+1)}, \boldsymbol{\beta}^{(t_{\boldsymbol{\beta}})}) \right\}^{-1} \frac{\partial}{\partial \boldsymbol{\beta}} l(\boldsymbol{\alpha}^{(t+1)} \boldsymbol{\beta}^{(t_{\boldsymbol{\beta}})})$
- **Step3c**: $t_{\boldsymbol{\beta}} \leftarrow t_{\boldsymbol{\beta}} + 1$
- **Step3d**: Do **Step3b** and **Step3c** iteratively until $\|\boldsymbol{\beta}^{(t_{\boldsymbol{\beta}})} - \boldsymbol{\beta}^{(t_{\boldsymbol{\beta}}-1)}\|_2 < 0.001$.
- **Step3e**: $\boldsymbol{\beta}^{(t+1)} \leftarrow \boldsymbol{\beta}^{(t_{\boldsymbol{\beta}}+1)}$

Step4: $t \leftarrow t + 1$

Step5: Do **Step2**, **Step3** and **Step4** iteratively until $\|(\boldsymbol{\alpha}^{(t)'}, \boldsymbol{\beta}^{(t)'})' - (\boldsymbol{\alpha}^{(t-1)'}, \boldsymbol{\beta}^{(t-1)'})'\|_2 < 10^{-6}$.

1.3 Implementation

```
minus.l = function(alpha, beta, dat){
  alpha0 = alpha[1]; alpha1 = alpha[2];
  beta0 = beta[1]; beta1 = beta[2];
  x = dat$lstat
  y = dat$medv.log
```

```
  return(-sum(-alpha0-alpha1*x-(1/(2*exp(alpha0+alpha1*x)))*(y-beta0-beta1*x)^2))
}

da1 = function(alpha, beta, dat){
  alpha0 = alpha[1]; alpha1 = alpha[2];
  beta0 = beta[1]; beta1 = beta[2];
  x = dat$lstat
  y = dat$medv.log
  n = nrow(dat)
  return(matrix(c(-n/2+sum((1/2)*exp(-alpha0-alpha1*x)*(y-beta0-beta1*x)^2),
    sum(-x/2+(1/2)*x*exp(-alpha0-alpha1*x)*(y-beta0-beta1*x)^2)), 2, 1))
}

da2 = function(alpha, beta, dat){
  alpha0 = alpha[1]; alpha1 = alpha[2];
  beta0 = beta[1]; beta1 = beta[2];
  x = dat$lstat
  y = dat$medv.log
  n = nrow(dat)
  return(matrix(c(sum(-(1/2)*exp(-alpha0-alpha1*x)*(y-beta0-beta1*x)^2),
           sum(-(1/2)*x*exp(-alpha0-alpha1*x)*(y-beta0-beta1*x)^2),
           sum(-(1/2)*x*exp(-alpha0-alpha1*x)*(y-beta0-beta1*x)^2),
           sum(-(1/2)*x^2*exp(-alpha0-alpha1*x)*(y-beta0-beta1*x)^2)), 2, 2))
}

db1 = function(alpha, beta, dat){
  alpha0 = alpha[1]; alpha1 = alpha[2];
  beta0 = beta[1]; beta1 = beta[2];
  x = dat$lstat
  y = dat$medv.log
  n = nrow(dat)
  return(matrix(c(sum(exp(-alpha0-alpha1*x)*(y-beta0-beta1*x)),
    sum(x*exp(-alpha0-alpha1*x)*(y-beta0-beta1*x))), 2, 1))
}

db2 = function(alpha, beta, dat){
  alpha0 = alpha[1]; alpha1 = alpha[2];
  beta0 = beta[1]; beta1 = beta[2];
  x = dat$lstat
  y = dat$medv.log
  n = nrow(dat)
  return(matrix(c(sum(-exp(-alpha0-alpha1*x)),
           sum(-x*exp(-alpha0-alpha1*x)),
           sum(-x*exp(-alpha0-alpha1*x)),
           sum(-x^2*exp(-alpha0-alpha1*x))), 2, 2))
}

norm1 = function(v){
  return(sum(abs(v)))
}

norm2 = function(v){
  return(sqrt(sum(v^2)))
```

```
}
```

```
gda = function(beta, dat){
  alpha = matrix(rep(0, 2), 2, 1)
  eps.conv = 0.001
  err = 1
  while(err > eps.conv){
    tmp = alpha - solve(-da2(alpha, beta, dat)) %*% -da1(alpha, beta, dat)
    err = norm2(tmp - alpha)
    alpha = tmp
  }
  return(alpha)
}

gdb = function(alpha, dat){
  beta = matrix(rep(0, 2), 2, 1)
  eps.conv = 0.001
  err = 1
  while(err > eps.conv){
    tmp = beta - solve(-db2(alpha, beta, dat)) %*% -db1(alpha, beta, dat)
    err = norm2(tmp - beta)
    beta = tmp
  }
  return(beta)
}
```

```
dat = read.csv("DataA.csv")
```

```
alpha = beta = matrix(rep(0, 2), 2, 1)
eps.conv = 0.001
err = 1
t = 1
while(err > eps.conv){
  alpha = cbind(alpha, gda(beta[,t], dat))
  beta = cbind(beta, gdb(alpha[,t+1], dat))
  t = t + 1
  err = norm2(c(alpha[,t], beta[,t]) - c(alpha[,t-1], beta[,t-1]))
}
```

```
alpha = beta = matrix(rep(0, 2), 2, 1)
eps.conv = 10^(-6)
err = 1
obj = matrix(c(minus.l(alpha, beta, dat)), 1, 1)
t = 1
while(err > eps.conv){
  alpha = cbind(alpha, gda(beta[,t], dat))
  beta = cbind(beta, gdb(alpha[,t+1], dat))
  t = t + 1
  obj = cbind(obj, minus.l(alpha[,t], beta[, t], dat))
  err = norm2(c(alpha[,t], beta[,t]) - c(alpha[,t-1], beta[,t-1]))
}
```

1.4 Results

Table 1 and Figure 1 show the trace for the algorithm. We can see that the algorithm indeed leads to a convergent result. For the final solution for each of the parameters and the minimized value of $-l(\boldsymbol{\alpha}, \boldsymbol{\beta})$, please see the last row in Table 1.

Table 1: Tabulation for the trace

	α_0	α_1	β_0	β_1	$-l(\boldsymbol{\alpha}, \boldsymbol{\beta})$
t = 0	0	0	0	0	2249.4488
t = 1	2.5713	-0.029	3.5368	-0.0412	1078.006
t = 2	-3.6701	0.048	3.6082	-0.0469	-1256.4138
t = 3	-3.8089	0.0563	3.6142	-0.0475	-1264.8166
t = 4	-3.82	0.0572	3.6148	-0.0475	-1264.8933
t = 5	-3.8211	0.0572	3.6149	-0.0475	-1264.8941
t = 6	-3.8212	0.0572	3.6149	-0.0475	-1264.8941
t = 7	-3.8213	0.0572	3.6149	-0.0475	-1264.8941
t = 8	-3.8213	0.0572	3.6149	-0.0475	-1264.8941
t = 9	**-3.8213**	**0.0572**	**3.6149**	**-0.0475**	**-1264.8941**

```
par(mfrow = c(3, 1))
matplot(0:(nrow(res)-1), t(alpha), col =1:nrow(alpha),
        xlab = "t", type = "l", ylab = "", lty = 1,
        main = expression(bold(alpha)))
legend("topright", c(expression(alpha[0]), expression(alpha[1])),
       lty = 1, col =1:nrow(alpha))
matplot(0:(nrow(res)-1), t(beta), col =1:nrow(beta),
        xlab = "t", type = "l", ylab = "", lty = 1,
        main = expression(bold(beta)))
legend("topright", c(expression(beta[0]), expression(beta[1])),
       lty = 1, col =1:nrow(beta))
matplot(0:(nrow(res)-1), t(obj), col =1:nrow(obj),
        xlab = "t", type = "l", ylab = "", lty = 1,
        main = expression(-l(bold(alpha), bold(beta))))
```

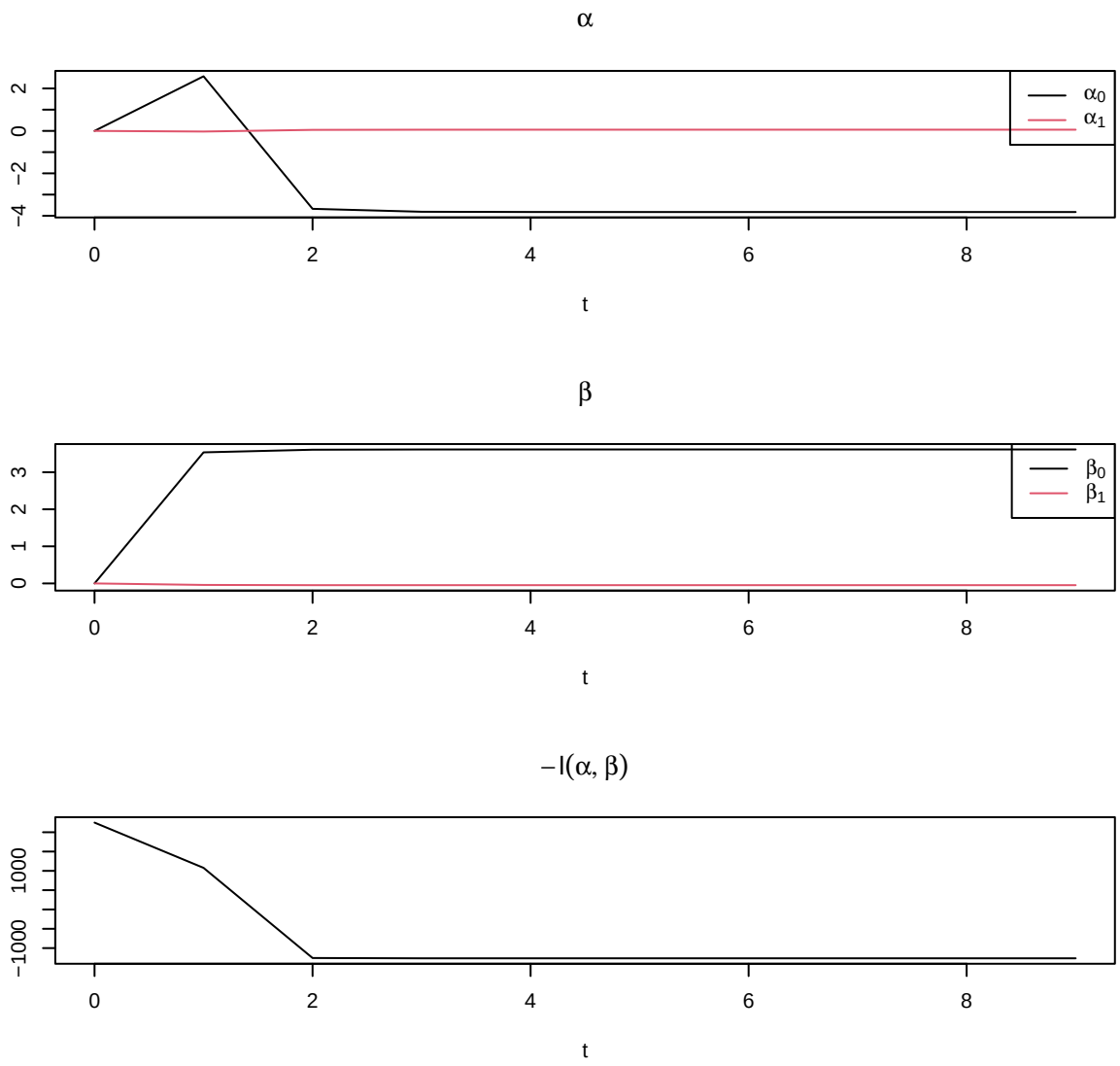

Figure 1: Visualization of the trace

2 Problem 2

2.1 Theories

Define $\boldsymbol{\beta} = (\beta_1, \dots, \beta_n)'$, $\delta_t \equiv (0, \dots, 0, 1, -2, \underbrace{1}_{t-\text{th}}, \dots, 0)\boldsymbol{\beta}$ for $t = 3, \dots, n$ and $\boldsymbol{\delta} = D\boldsymbol{\beta}$,

where

$$D = \begin{bmatrix} 1 & -2 & 1 & \cdots & 0 \\ 0 & 1 & -2 & 1 & \cdots \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & \cdots & 1 & -2 & 1 \end{bmatrix}$$

Let

$$f(\boldsymbol{\beta}) = \frac{1}{2}\|\boldsymbol{y} - \boldsymbol{\beta}\|_2^2,$$

and

$$g(\boldsymbol{\delta}) = \tau\|\boldsymbol{\delta}\|_1.$$

We can reformulate the problem as

$$\min\{f(\boldsymbol{\beta}) + g(\boldsymbol{\delta})\} \quad \text{subject to} \quad D\boldsymbol{\beta} - \boldsymbol{\delta} = \mathbf{0}.$$

where $f(\cdot)$ and $g(\cdot)$ are convex. It is clear that we can solve this problem by Alternating Direction Method of Multiplier (ADMM).

Define

$$L_\rho(\boldsymbol{\beta}, \boldsymbol{\delta}, \boldsymbol{\lambda}) \equiv f(\boldsymbol{\beta}) + g(\boldsymbol{\delta}) + \boldsymbol{\lambda}'(D\boldsymbol{\beta} - \boldsymbol{\delta}) + \frac{\rho}{2}\|D\boldsymbol{\beta} - \boldsymbol{\delta}\|_2^2,$$

where $\boldsymbol{\lambda} = (\lambda_1, \dots, \lambda_{98})'$ is an $(n-2)$-dimensional vector and ρ is a scalar. In ADMM, we need to know how to solve the following sub-problems:

1. Minimize $L_\rho(\boldsymbol{\beta}, \boldsymbol{\delta}, \boldsymbol{\lambda})$ over $\boldsymbol{\beta}$ when the other parameters are held fixed.
2. Minimize $L_\rho(\boldsymbol{\beta}, \boldsymbol{\delta}, \boldsymbol{\lambda})$ over $\boldsymbol{\delta}$ when the other parameters are held fixed.
3. Make $L_\rho(\boldsymbol{\beta}, \boldsymbol{\delta}, \boldsymbol{\lambda})$ larger by changing $\boldsymbol{\lambda}$ while holding the other parameters fixed.

In the following, we call these three sub-problems by **P1**, **P2** and **P3** respectively.

Solution for P1

Treating $\boldsymbol{\delta}$ and $\boldsymbol{\lambda}$ as fixed, we derive $\arg\min_{\boldsymbol{\beta}} L_\rho(\boldsymbol{\beta}, \boldsymbol{\delta}, \boldsymbol{\lambda})$ as follows.

$$\frac{\partial}{\partial \boldsymbol{\beta}'} L_\rho(\boldsymbol{\beta}, \boldsymbol{\delta}, \boldsymbol{\lambda}) = (\boldsymbol{\beta} - \boldsymbol{y}) + D'\boldsymbol{\lambda} + \rho D'(D\boldsymbol{\beta} - \boldsymbol{\delta}),$$

where I is an $n \times n$ identity matrix. We let this equation be zero, and derive that

$$arg\min_{\boldsymbol{\beta}} L_\rho(\boldsymbol{\beta},\boldsymbol{\delta},\boldsymbol{\lambda}) = (I+\rho D'D)^{-1}[\boldsymbol{y}+\rho D'(\boldsymbol{\delta}-\boldsymbol{\lambda}/\rho)].$$

Solution for P2

Treating $\boldsymbol{\beta}$ and $\boldsymbol{\lambda}$ as fixed, we derive $\arg\min_{\boldsymbol{\delta}} L_\rho(\boldsymbol{\beta},\boldsymbol{\delta},\boldsymbol{\lambda})$ as follows. Before the derivation, we first note that

$$\arg\min_{\boldsymbol{\delta}} L_\rho(\boldsymbol{\beta},\boldsymbol{\delta},\boldsymbol{\lambda}) \equiv f(\boldsymbol{\beta}) + g(\boldsymbol{\delta}) + \boldsymbol{\lambda}'(D\boldsymbol{\beta}-\boldsymbol{\delta}) + \frac{\rho}{2}\|D\boldsymbol{\beta}-\boldsymbol{\delta}\|_2^2 = \arg\min_{\boldsymbol{\delta}} \frac{1}{2}\|\boldsymbol{\delta}-(D\boldsymbol{\beta}+\boldsymbol{\lambda}/\rho)\|_2^2 + \frac{\tau}{\rho}\|\boldsymbol{\delta}\|_1.$$

Therefore, it follows that

$$\frac{\partial}{\partial\boldsymbol{\delta}'}\left[\frac{1}{2}\|\boldsymbol{\delta}-(D\boldsymbol{\beta}+\boldsymbol{\lambda}/\rho)\|_2^2 + \frac{\tau}{\rho}\|\boldsymbol{\delta}\|_1\right] = \{\boldsymbol{\delta}-(D\boldsymbol{\beta}+\boldsymbol{\lambda}/\rho)\} + \frac{\tau}{\rho}\text{sign}(\boldsymbol{\delta}).$$

We let this equation be zero, and derive that

$$arg\min_{\boldsymbol{\delta}} L_\rho(\boldsymbol{\beta},\boldsymbol{\delta},\boldsymbol{\lambda}) = \text{sign}(D\boldsymbol{\beta}+\boldsymbol{\lambda}/\rho)\left(|D\boldsymbol{\beta}+\boldsymbol{\lambda}/\rho| - \frac{\tau}{\rho}\mathbf{1}\right)_+,$$

where $\mathbf{1}$ is an $(n-2)$-dimensional vector with all of 1.

Solution for P3

Treating $\boldsymbol{\beta}$ and $\boldsymbol{\delta}$ as fixed, we have

$$\frac{\partial}{\partial\boldsymbol{\lambda}'} L_\rho(\boldsymbol{\beta},\boldsymbol{\delta},\boldsymbol{\lambda}) = D\boldsymbol{\beta}-\boldsymbol{\delta}.$$

To make $L_\rho(\boldsymbol{\beta},\boldsymbol{\delta},\boldsymbol{\lambda})$ larger, it is reasonable to do the following change on $\boldsymbol{\lambda}$:

$$\boldsymbol{\lambda} \leftarrow \boldsymbol{\lambda} + \rho(D\boldsymbol{\beta}-\boldsymbol{\delta}).$$

We are now able to write the algorithm of ADMM.

2.2 Algorithm

Step1: Set $\tau = 1$ and $\rho = 0.01$. For $t = 0$, initialize $\boldsymbol{\beta}^{(t)} = \bar{y}\mathbf{1}_{100}$ and $\boldsymbol{\delta}^{(t)} = 5\mathbf{1}_{98}$, , where $\mathbf{1}_n = (\overbrace{1,\dots,1}^{n's})'$

Step2: $\boldsymbol{\beta}^{(t+1)} \leftarrow (I+\rho D'D)^{-1}[\boldsymbol{y}+\rho D'(\boldsymbol{\delta}^{(t)}-\boldsymbol{\lambda}^{(t)}/\rho)]$

Step3: $\boldsymbol{\delta}^{(t+1)} \leftarrow \text{sign}(D\boldsymbol{\beta}^{(t+1)}+\boldsymbol{\lambda}^{(t)}/\rho)\left(|D\boldsymbol{\beta}^{(t+1)}+\boldsymbol{\lambda}^{(t)}/\rho| - \frac{\tau}{\rho}\mathbf{1}\right)_+$

Step4: $\boldsymbol{\lambda}^{(t+1)} \leftarrow \boldsymbol{\lambda}^{(t)} + \rho(D\boldsymbol{\beta}^{(t+1)}-\boldsymbol{\delta}^{(t+1)})$

Step5: $t \leftarrow t+1$

Step6: Do **Step2**, **Step3**, **Step4** and **Step5** iteratively until

$$\max\{|(\boldsymbol{\beta}^{(t)'},\boldsymbol{\delta}^{(t)'},\boldsymbol{\lambda}^{(t)'})' - (\boldsymbol{\beta}^{(t-1)'},\boldsymbol{\delta}^{(t-1)'},\boldsymbol{\lambda}^{(t-1)'})'|\} < 0.001.$$

2.3 Implementation

```r
y = read_csv("DataB.csv")$x
n = length(y)
```

```r
rho = 0.01
tau = 1
D = matrix(NA, (n-2), n)
for(i in 1:(n-2)){
  D[i,] = c(rep(0, i-1), c(1, -2, 1), rep(0, (n-i-2)))
}
I = diag(n)
```

```r
update.beta = function(delta, lambda, y){
  beta = solve(I+rho*t(D)%*%D)%*%(y+rho*t(D)%*%(delta-lambda/rho))
  return(beta)
}

update.delta = function(beta, lambda, y){
  delta = sign(D%*%beta+lambda/rho)*sapply(abs(D%*%beta+lambda/rho)-tau/rho, max, 0)
  return(delta)
}

update.lambda = function(beta, delta, lambda, y){
  lambda1 = lambda+rho*(D%*%beta-delta)
  return(lambda1)
}

L = function(beta, delta, tau, y){
  return((1/2)*norm2(y-beta)^2+tau*norm1(delta))
}
```

```r
beta = matrix(rep(mean(y), n), n, 1)
delta = lambda =  matrix(rep(5, n-2), n-2, 1)
obj = matrix(c(L(beta, delta, tau, y)), 1, 1)
eps.conv = 0.001
err = 1
t = 1
while(err > eps.conv){
  beta = cbind(beta, update.beta(delta[,t], lambda[,t], y))
  delta = cbind(delta, update.delta(beta[,t+1], lambda[,t], y))
  lambda = cbind(lambda, update.lambda(beta[,t+1], delta[,t+1], lambda[,t], y))
  t = t + 1
  obj = cbind(obj, L(beta[,t], delta[,t], tau, y))
  err = max(abs(c(beta[,t], delta[,t], lambda[,t]) - c(beta[,t-1], delta[,t], lambda[,t-1])))
}
```

2.4 Results

Figure 2 shows the trace for the algorithm. We can see that the algorithm leads to a convergent result. For the final solution for $\boldsymbol{\beta}$, $\boldsymbol{\lambda}$, and $\boldsymbol{\delta}$ please see Table 2, Table 3 and Table 4. Eventually, it is found that the minimized value of $f(\boldsymbol{\beta}) + g(\boldsymbol{\delta})$ is 39.0151.

Table 2: Final solution for$\boldsymbol{\beta}$

β_{1}	β_{2}	β_{3}	β_{4}	β_{5}	β_{6}	β_{7}	β_{8}	β_{9}	β_{10}	β_{11}	β_{12}	β_{13}	β_{14}	β_{15}	β_{16}	β_{17}	β_{18}	β_{19}	β_{20}
-0.44	-0.32	-0.2	-0.09	0	0.1	0.25	0.41	0.57	0.23	-0.11	-0.45	-0.37	0.01	0.35	0.62	0.79	0.87	0.9	0.51
β_{21}	β_{22}	β_{23}	β_{24}	β_{25}	β_{26}	β_{27}	β_{28}	β_{29}	β_{30}	β_{31}	β_{32}	β_{33}	β_{34}	β_{35}	β_{36}	β_{37}	β_{38}	β_{39}	β_{40}
0.11	-0.31	-0.74	-1.01	-0.79	-0.57	-0.36	-0.15	0.05	1.05	2.29	3.54	3.73	3.9	3.99	3.99	3.89	3.73	3.54	3.56
β_{41}	β_{42}	β_{43}	β_{44}	β_{45}	β_{46}	β_{47}	β_{48}	β_{49}	β_{50}	β_{51}	β_{52}	β_{53}	β_{54}	β_{55}	β_{56}	β_{57}	β_{58}	β_{59}	β_{60}
3.59	3.62	3.65	3.67	3.68	3.76	3.83	3.96	4.07	4.16	4.23	4.27	4.28	4.25	4.18	4.02	3.79	3.49	3.16	2.14
β_{61}	β_{62}	β_{63}	β_{64}	β_{65}	β_{66}	β_{67}	β_{68}	β_{69}	β_{70}	β_{71}	β_{72}	β_{73}	β_{74}	β_{75}	β_{76}	β_{77}	β_{78}	β_{79}	β_{80}
1.13	0.14	-0.44	-0.4	-0.37	-0.35	-0.35	-0.36	-0.16	0.09	0.33	0.58	0.43	0.28	0.24	0.19	0.11	0.01	-0.1	-0.06
β_{81}	β_{82}	β_{83}	β_{84}	β_{85}	β_{86}	β_{87}	β_{88}	β_{89}	β_{90}	β_{91}	β_{92}	β_{93}	β_{94}	β_{95}	β_{96}	β_{97}	β_{98}	β_{99}	β_{100}
0.02	0.09	0.17	0.24	0.29	0.31	0.29	0.23	0.12	-0.02	-0.17	-0.24	-0.31	-0.38	-0.4	-0.43	-0.45	-0.01	0.44	0.88

```
par.names = paste("$\\delta_{", 3:100, "}$", sep = "")
res = rbind(par.names[1:20], round(delta[1:20, t], 2),
            par.names[21:40], round(delta[21:40, t], 2),
            par.names[41:60], round(delta[41:60, t], 2),
            par.names[61:80], round(delta[61:80, t], 2),
            c(par.names[81:98], "", ""), c(round(delta[81:98, t], 2), "", ""))

kable(res,
      align = "c", booktabs = TRUE, linesep = "", escape = FALSE,
      caption = "Final solution for$\\pmb{\\delta}$") %>%
  kable_styling(latex_options = c("striped", "hold_position", "scale_down")) %>%
  row_spec(1:9, hline_after = TRUE)
```

Table 3: Final solution for$\boldsymbol{\delta}$

δ_{3}	δ_{4}	δ_{5}	δ_{6}	δ_{7}	δ_{8}	δ_{9}	δ_{10}	δ_{11}	δ_{12}	δ_{13}	δ_{14}	δ_{15}	δ_{16}	δ_{17}	δ_{18}	δ_{19}	δ_{20}	δ_{21}	δ_{22}
0	0	0	0	0.06	0	0	-0.5	0	0	0.42	0.3	0	0	0	0	0	-0.43	0	0
δ_{23}	δ_{24}	δ_{25}	δ_{26}	δ_{27}	δ_{28}	δ_{29}	δ_{30}	δ_{31}	δ_{32}	δ_{33}	δ_{34}	δ_{35}	δ_{36}	δ_{37}	δ_{38}	δ_{39}	δ_{40}	δ_{41}	δ_{42}
0	0.16	0.49	0	0	0	0	0.8	0.24	0	-1.05	0	0	0	0	0	0	0.21	0	0
δ_{43}	δ_{44}	δ_{45}	δ_{46}	δ_{47}	δ_{48}	δ_{49}	δ_{50}	δ_{51}	δ_{52}	δ_{53}	δ_{54}	δ_{55}	δ_{56}	δ_{57}	δ_{58}	δ_{59}	δ_{60}	δ_{61}	δ_{62}
0	0	0	0.05	0	0.05	0	0	0	0	0	0	0	0	0	0	0	-0.68	0	0
δ_{63}	δ_{64}	δ_{65}	δ_{66}	δ_{67}	δ_{68}	δ_{69}	δ_{70}	δ_{71}	δ_{72}	δ_{73}	δ_{74}	δ_{75}	δ_{76}	δ_{77}	δ_{78}	δ_{79}	δ_{80}	δ_{81}	δ_{82}
0.42	0.61	0	0	0	0	0.2	0.05	0	0	-0.4	0	0.11	0	0	0	0	0.15	0.03	0
δ_{83}	δ_{84}	δ_{85}	δ_{86}	δ_{87}	δ_{88}	δ_{89}	δ_{90}	δ_{91}	δ_{92}	δ_{93}	δ_{94}	δ_{95}	δ_{96}	δ_{97}	δ_{98}	δ_{99}	δ_{100}		
0	0	0	0	0	0	0	0	0	0.08	0	0	0.05	0	0	0.47	0	0		

```
par(mfrow = c(4, 1))
matplot(0:(ncol(beta)-1), t(beta), col =1:nrow(beta),
        xlab = "t", type = "l", ylab = "", lty = 1,
        main = expression(bold(beta)))
matplot(0:(ncol(delta)-1), t(delta), col =1:nrow(delta),
        xlab = "t", type = "l", ylab = "", lty = 1,
        main = expression(bold(delta)))
matplot(0:(ncol(lambda)-1), t(lambda), col =1:nrow(lambda),
        xlab = "t", type = "l", ylab = "", lty = 1,
        main = expression(bold(lambda)))
```

Table 4: Final solution for$\boldsymbol{\lambda}$

λ_1	λ_2	λ_3	λ_4	λ_5	λ_6	λ_7	λ_8	λ_9	λ_{10}	λ_{11}	λ_{12}	λ_{13}	λ_{14}	λ_{15}	λ_{16}	λ_{17}	λ_{18}	λ_{19}	λ_{20}
-0.13	0.29	0.41	0.75	1	0.89	-0.53	-1	0.24	-0.01	1	1	0.62	-0.06	-0.81	-0.11	-0.79	-1	0.02	-0.29
λ_{21}	λ_{22}	λ_{23}	λ_{24}	λ_{25}	λ_{26}	λ_{27}	λ_{28}	λ_{29}	λ_{30}	λ_{31}	λ_{32}	λ_{33}	λ_{34}	λ_{35}	λ_{36}	λ_{37}	λ_{38}	λ_{39}	λ_{40}
0.49	1	1	0.42	0.17	0.31	0.92	1	1	-0.08	-1	-0.72	-0.06	-0.95	-0.92	0.25	0.67	1	0.36	0.04
λ_{41}	λ_{42}	λ_{43}	λ_{44}	λ_{45}	λ_{46}	λ_{47}	λ_{48}	λ_{49}	λ_{50}	λ_{51}	λ_{52}	λ_{53}	λ_{54}	λ_{55}	λ_{56}	λ_{57}	λ_{58}	λ_{59}	λ_{60}
0.55	0.23	0.97	1	0.82	1	0.87	-0.03	-0.23	0.08	0.57	0.95	0.01	-0.27	0.05	-0.15	-0.44	-1	-0.64	0.37
λ_{61}	λ_{62}	λ_{63}	λ_{64}	λ_{65}	λ_{66}	λ_{67}	λ_{68}	λ_{69}	λ_{70}	λ_{71}	λ_{72}	λ_{73}	λ_{74}	λ_{75}	λ_{76}	λ_{77}	λ_{78}	λ_{79}	λ_{80}
1	1	0.47	-0.34	0.12	0.53	1	1	0.92	-0.29	-1	-0.42	1	0.39	-0.71	-0.93	0.24	1	1	0.79
λ_{81}	λ_{82}	λ_{83}	λ_{84}	λ_{85}	λ_{86}	λ_{87}	λ_{88}	λ_{89}	λ_{90}	λ_{91}	λ_{92}	λ_{93}	λ_{94}	λ_{95}	λ_{96}	λ_{97}	λ_{98}		
0.72	0.32	0.71	0.88	0.78	-0.47	-0.24	0.42	0.51	1	0.87	0.36	1	0.96	0.73	1	0.45	0.08		

```
matplot(0:(ncol(obj)-1), t(obj), col =1:nrow(obj),
        xlab = "t", type = "l", ylab = "", lty = 1,
        main = expression(f(bold(beta))+g(bold(delta))))
```

Figure 3 shows $\hat{\beta}_t$ against to y_t for $t = 1, \ldots, 100$. We can see that our solution for$\boldsymbol{\beta}$ is quite satisfactory from the perspective of fitting.

```
ts.plot(cbind(y,beta[,t]), col = 1:2)
```

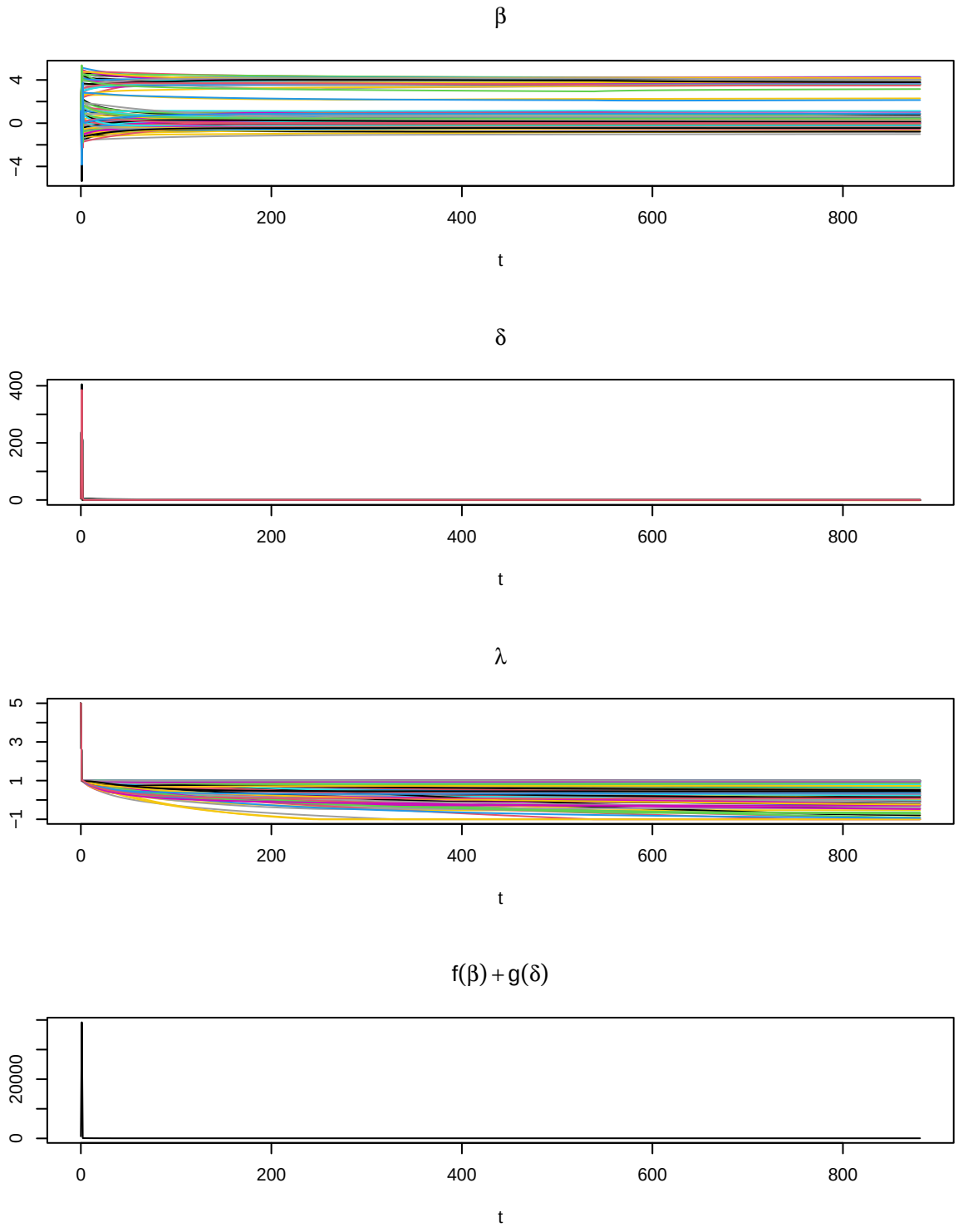

Figure 2: Visualization of the trace

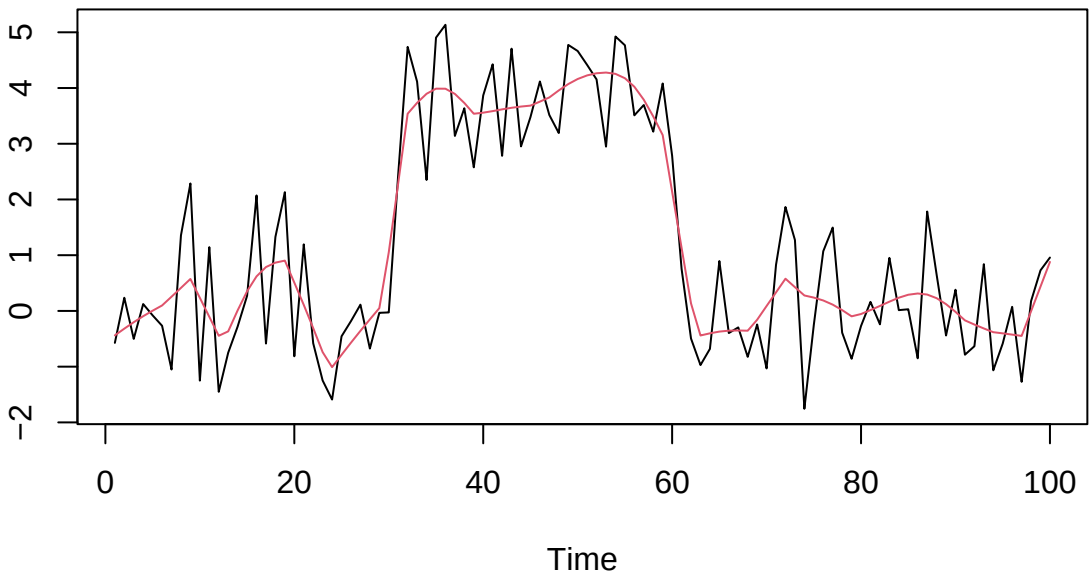

Figure 3: $\hat{\beta}_t$ versus y_t

3 Problem 3

3.1 Theories

Let $f_j(x) = -\sin(\pi x)[\sin(j\pi x^2)]^{20}$. We have $f(\boldsymbol{x}) = \sum_{j=1}^{10} f_j(x_j)$. Figure 5 in Appendix A.1 shows the functional shape of $f_j(x)$ for $j = 1, \dots, 10$. It is clear that all the $f_j(x)$'s except for $f_1(x)$ are non-convex functions. Hence, it is possible to stuck in a local minimum of $f(\boldsymbol{x})$ when we are attempt to find the global minimum. Moreover, we note that there is no cross-product term of x_i and x_k for $i \neq k$. Therefore, we will address the problem by minimizing $f_j(x_j)$ for $j = 1, \dots, 10$.

For $j = 1, \dots, 10$, let $g_j = \frac{\partial f(\boldsymbol{x})}{\partial x_j}$. We have

$$g_j(x_j) = -\pi[\sin(j\pi x_j^2)]^{19}\{\cos(\pi x_j)\sin(j\pi x_j^2) + 40jx_j\sin(\pi x_j)\cos(j\pi x_j^2)\}, \quad j = 1, \dots, 10.$$

Define $g(\boldsymbol{x}) = (g_1, \dots, g_{10})'$, which is the gradient of $f(\boldsymbol{x})$.

Recall the risk of stucking in a local minimum. It is reasonable to think of a gradient-descent based method with the concept of momentum added throughout the algorithm. With a momentum term, we can give the update process a tendency to keep moving in the same direction. The algorithm is detailed as follows.

Furthermore, from Figure 5 we can see there are many local minimums, which may let the algorithm be sensitive to the initial values. Hence, we would try different initial values in $\{0, 0.01, \dots, 1\}$, and choose the one that provides the optimal result.

3.2 Algorithm

Let α be the learning rate, γ be the momentum factor and v_j be the speed of x_j for $j = 1, \dots, 10$. For $j = 1, \dots, 10$, let $x_{0,j}$ be an initial value for x_j.

Recall that our strategy is to optimize $f_j(x_j)$ separately for $j = 1, \dots, 10$. We accomplish it as follows.

Step1: Set $\alpha = 10^{-4}$ and $\gamma = 0.9$. For $t = 0$, initialize $x_j^{(t)} = x_{0,j}$ and $v_j^{(t)} = 0.02$.

Step2: $v_j^{(t+1)} \leftarrow \gamma v_j^{(t)} - \alpha g_j(x_j^{(t)})$

Step3: $x_j^{(t+1)} \leftarrow x_j^{(t)} + v_j^{(t+1)}$

Step4: $t \leftarrow t + 1$

Step5: Do **Step2**, **Step3** and **Step4** iteratively until $t = 1000$.

For each $x_{0,j}$ in $\{0, 0.01, \dots, 1\}$, we do **Step1** $\sim$ **Step5**. Therefore, there will be 101 solutions for x_j. We then pick the most optimal one which makes $f_j(x)$ the smallest.

3.3 Implementation

```
f = function(x){
  return(-sum(sin(pi*x)*sin((1:10)*pi*x^2)^20))
}

g = function(x, j){
  grad = rep(NA, 10)
  for(j in 1:10){
    grad[j] = (-pi*(sin(j*pi*x[j]^2))^(19)*(cos(pi*x[j])*sin(j*pi*x[j]^2)
                                +40*j*x[j]*sin(pi*x[j])*cos(j*pi*x[j]^2)))
  }
  return(grad)
}
```

```
gdm = function(init, alpha, gamma, n.iter, f, g){
  x = matrix(init, length(init), 1)
  v = matrix(rep(0.02, length(init)), length(init), 1)
  obj = c(f(x))
  err = 1
  for(t in 1:n.iter){
    grad = g(x[,t])
    vt = gamma*v[,t]-alpha*(grad)
    xt = x[,t]+vt
    if(all(xt > 0 & xt < 1)){
      v = cbind(v, vt)
      x = cbind(x, xt)
      obj = c(obj, f(x[,t]))
      err = max(abs(x[,t+1]-x[,t]))
    }else{
      v = cbind(v, v[,t])
      x = cbind(x, x[,t])
    }
  }
```

```
  return(list(x = x, v = v, obj = obj, time = n.iter))
}
```

```
n.iter = 1000
x = matrix(NA, 10, n.iter+1)
obj = matrix(NA, 10, n.iter+1)
x0.v = c()
for(j in 1:10){
  x0 = seq(0, 1, by = 0.01)
  obj.values = rep(NA, length(x0))
  for(i in 1:length(x0)){
    print(i)
    opt = gdm(rep(x0[i], 1), alpha = 10^(-4), gamma = 0.9,
              n.iter, function(x) fj(x, j), function(x) gj(x, j))
    obj.values[i] = opt$obj[opt$time]
  }
  opt.i = which.min(obj.values)
  x0.v[j] = x0[opt.i]
  opt = gdm(rep(x0[opt.i], 1), alpha = 10^(-4), gamma = 0.9,
            n.iter, function(x) fj(x, j), function(x) gj(x, j))
  xj = opt$x
  objj = matrix(opt$obj, nrow = 1)
  t = opt$time
  x[j,] = xj
  obj[j,] = objj
}
obj = matrix(apply(obj, 2, sum), 1, n.iter+1)
dump(c("x0.v", "x", "obj"), "problem3.Rdata")
```

3.4 Results

Figure 4 shows the trace for the algorithm with $\boldsymbol{x}_0$ shown in Table 5. We can see that the algorithm leads to a convergent result. For the final solution for $\boldsymbol{x}$, please see Table 6. Eventually, it is found that the minimized value of $f(\boldsymbol{x})$ is -9.6602.

Table 5: Initial values for $\boldsymbol{x}_0$

$x_{0,1}$	$x_{0,2}$	$x_{0,3}$	$x_{0,4}$	$x_{0,5}$	$x_{0,6}$	$x_{0,7}$	$x_{0,8}$	$x_{0,9}$	$x_{0,10}$
0.32	0.16	0.11	0.4	0.35	0.31	0.3	0.38	0.34	0.32

```
par(mfrow = c(2, 1))
matplot(0:(ncol(x)-1), t(x), col =1:nrow(x),
        xlab = "t", type = "l", ylab = "", lty = 1,
        main = expression(bold(x)),
        cex.main = 0.8, cex.lab = 0.8, cex.axis = 0.8)
matplot(0:(ncol(obj)-1), t(obj), col =1,
        xlab = "t", type = "l", ylab = "", lty = 1,
        main = expression(f(bold(x))),
        cex.main = 0.8, cex.lab = 0.8, cex.axis = 0.8)
```

We also provide two alternative approaches for this problem in Appendix A.2 and A.3 respectively. Due to the time limitation, the details for Appendix A.3 are omitted temporarily.

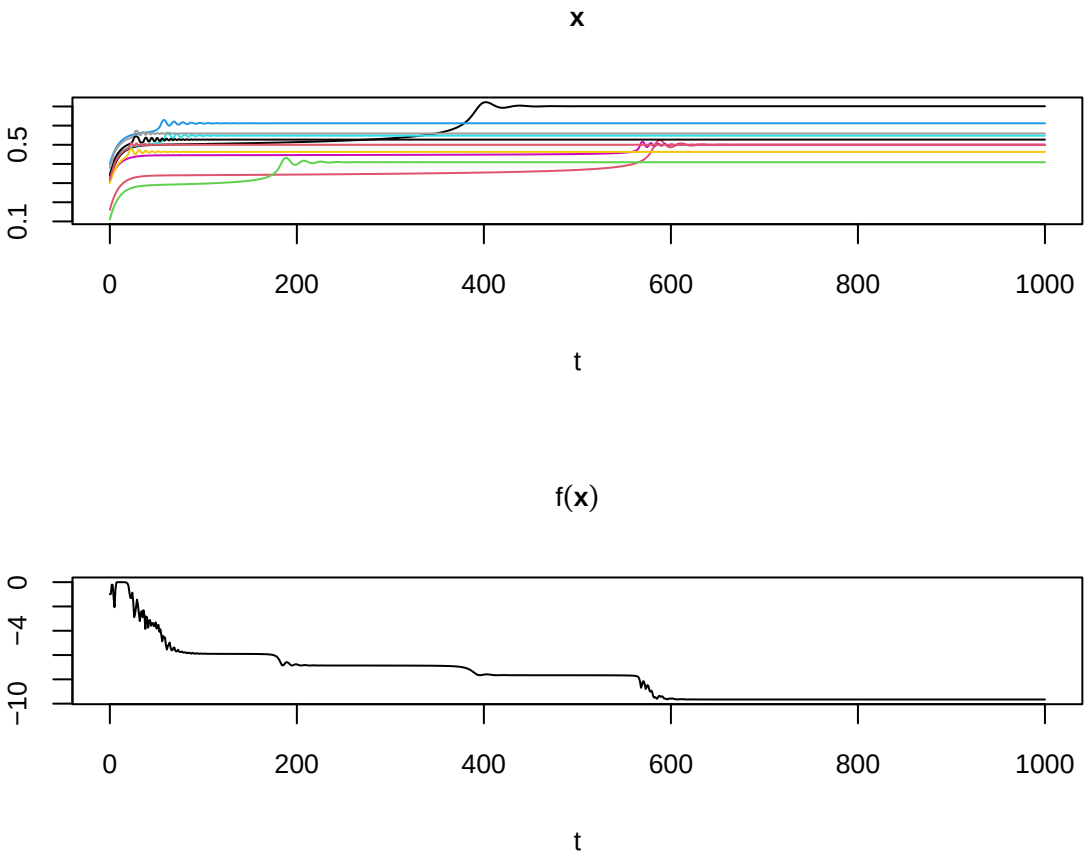

Figure 4: Visualization of the trace

Table 6: Final solution for $\boldsymbol{x}$

x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}
0.7012	0.5	0.409	0.6121	0.5476	0.5	0.463	0.559	0.527	0.5

4 Appendix

4.1 A.1

Please see Figure 5.

```
fj = function(x, j){
  v = rep(NA, length(x))
  for(i in 1:length(x)){
    v[i] = -sin(pi*x[i])*sin(j*pi*x[i]^2)^20
  }
  return(v)
}
```

```
par(mfrow = c(5, 2))
x = seq(0, 1, len = 100)
for(j in 1:10){
  plot(x, fj(x, j), type = "l", ylab = bquote(f[.(j)](x)))
}
```

4.2 A.2 (Simulated Annealing)

Since there is no cross-product term of x_i and x_k for $i \neq k$, we can reduce the problem to minimization of $f_j(x_j)$ for $j = 1, \dots, 10$.

Simulated Annealing (SA) is a well-known method for dealing with such non-convex optimization problem. Moreover, we note that there is no cross-product term of x_i and x_k for $i \neq k$. Therefore, we might be able to address the problem by minimizing $f_j(x_j)$ for $j = 1, \dots, 10$ using SA. The algorithm is detailed as follows.

4.2.1 Algorithm

Let T be the temperature and R be the cool down factor. For $j = 1, \dots, 10$,

Step1: Set $R_t = 0.95$. For $t = 0$, initialize $T^{(t)} = 1$ and $x_j^{(t)} = \overbrace{1 \dots 1}^{10\,\text{ones}}$ with binary coding.

Step2:

> **Step2a**: Generate i from $\{1, \dots, 10\}$ randomly with equal probability.
>
> **Step2b**: Switch the ith digit of $x_j^{(t)}$ to the opposite (i.e., either $0 \to 1$ or $1 \to 0$).
>
> **Step2c**: Let x^* be this new number.

Step3:

> **Step3a**: Calculate $\Delta f_j = f_j(x_j^{(t)}) - f_j(x^*)$.
>
> **Step3b**:

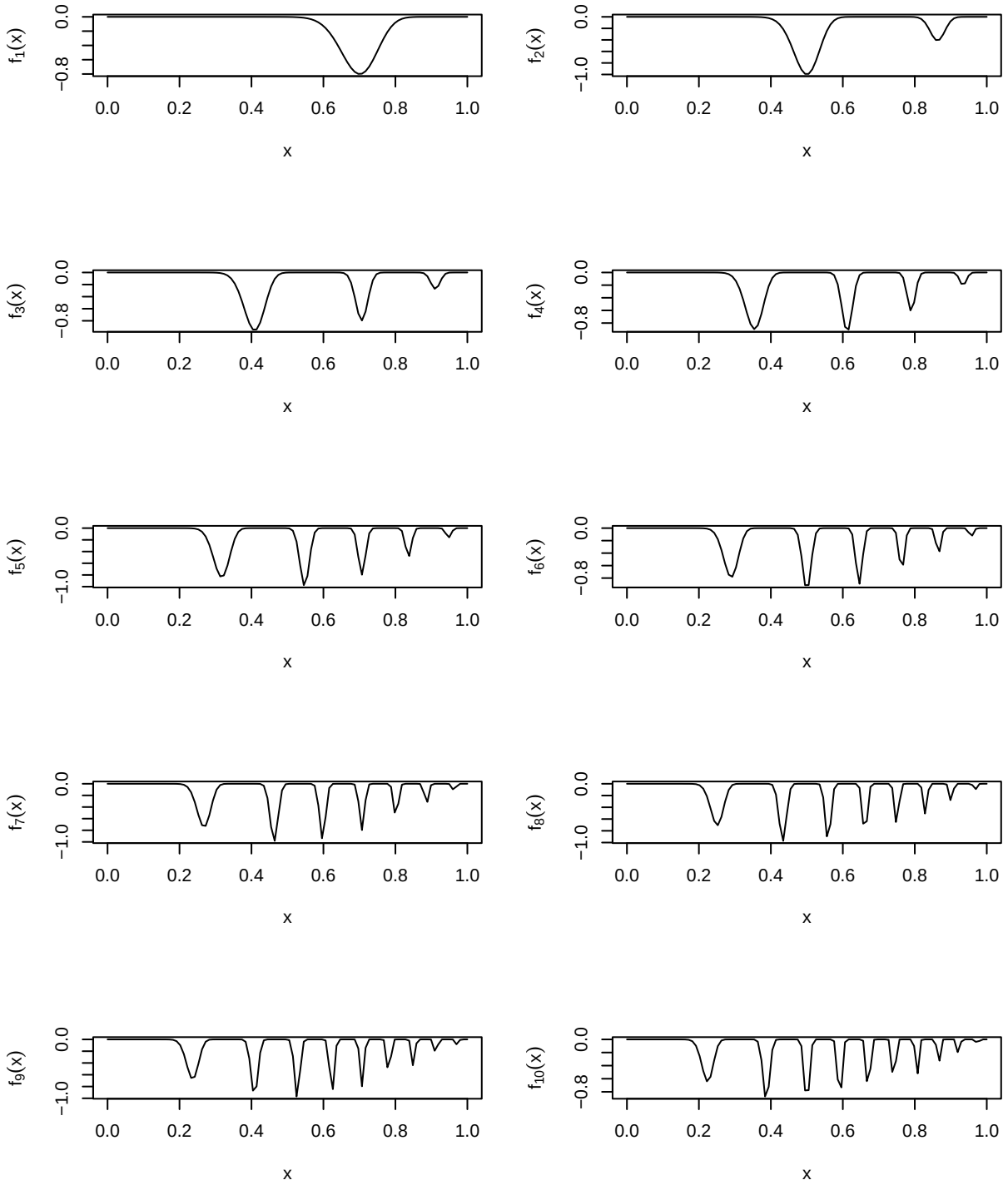

Figure 5: $f_j(x)$, $j = 1, \ldots, 10$

$$\begin{cases} x_j^{(t+1)} \leftarrow x^* \text{ and go to } \textbf{Step5}, & \text{if } \Delta f \geq 0 \\ \text{Go to } \textbf{Step4}, & \text{o.w.} \end{cases}$$

Step4:

Step4a: Caluculate acceptance rate $r = e^{\Delta f / T^{(t)}}$.

Step4b: Generate $U \sim \mathcal{U}niform(0, 1)$.

Step4c:

$$\begin{cases} x_j^{(t+1)} \leftarrow x^*, & \text{if } U \leq r \\ x_j^{(t+1)} \leftarrow x_j^{(t)}, & \text{o.w.} \end{cases}$$

Step5: $T^{(t+1)} \leftarrow T^{(t)} \times R$

Step6: $t \leftarrow t + 1$

Step7: Do **Step2** $\sim$ **Step6** iteratively until $t = 10000$.

4.2.2 Implementation

```
unbinary = function(x){
  return(sum(2^((-1):(-10))*x))
}
transition = function(x){
  i = sample(1:10, 1)
  x[i] = ifelse(x[i] == 1, 0, 1)
  return(x)
}
```

```
set.seed(111024513)
n.iter = 10000
x = obj = matrix(NA, 10, (n.iter+1))
for(j in 1:10){
  Temp = 1
  R = 0.95
  xj = 0.5
  objj = c(fj(xj, j))
  for(t in 1:n.iter){
    tmp.x = xj[t] + rnorm(1, 0, 0.001)
    if(tmp.x > 0 & tmp.x <1){
      if(objj[t]-fj(tmp.x, j) >= 0){
        xj = c(xj, tmp.x)
      }else{
         r = exp((objj[t]-fj(tmp.x, j))/Temp)
         if(runif(1) <= r){
           xj = c(xj, tmp.x)
         }else{
           xj = c(xj, xj[t])
         }
      }
```

```
    }else{
      xj = c(xj, xj[t])
    }
    objj = c(objj, fj(xj[t+1], j))
    Temp = Temp*R
  }
  obj[j,] = objj
  x[j,] = xj
}
obj = matrix(apply(obj, 2, sum), 1, n.iter+1)
dump(c("x", "obj"), "appendix_A2.RData")
```

4.2.3 Results

Figure 6 shows the trace for the algorithm. We can see that the algorithm leads to a convergent result. For the final solution for $\boldsymbol{x}$, please see Table 7. Eventually, it is found that the minimized value of $f(\boldsymbol{x})$ is -9.3417.

```
par(mfrow = c(2, 1))
matplot(0:(ncol(x)-1), t(x), col =1:nrow(x),
        xlab = "t", type = "l", ylab = "", lty = 1,
        main = expression(bold(x)),
        cex.main = 0.8, cex.lab = 0.8, cex.axis = 0.8)
matplot(0:(ncol(obj)-1), t(obj), col =1,
        xlab = "t", type = "l", ylab = "", lty = 1,
        main = expression(f(bold(x))),
        cex.main = 0.8, cex.lab = 0.8, cex.axis = 0.8)
```

Table 7: Final solution for $\boldsymbol{x}$

x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}
0.7012	0.499	0.707	0.6123	0.5459	0.499	0.5977	0.4375	0.5273	0.499

4.3 A.3 (A Variant of Particle Swarm Optimization)

4.3.1 Implementation

```
pso.sgdm = function(s, n.iter, alpha, gamma){
  x = matrix(runif(s*10), 10, s)
  v = matrix(runif(s*10, 0.009, 0.011), 10, s)
  p = x
  x.opt = matrix(x[,which.max(apply(p, 2, f))], 10, 1)
  obj = c(f(x.opt))

  update.par = function(x, v, alpha, gamma){
    grad = g(x)
    vt = gamma*v-alpha*(grad)
    xt = x+vt
    if(all(xt > 0 & xt < 1)){
```

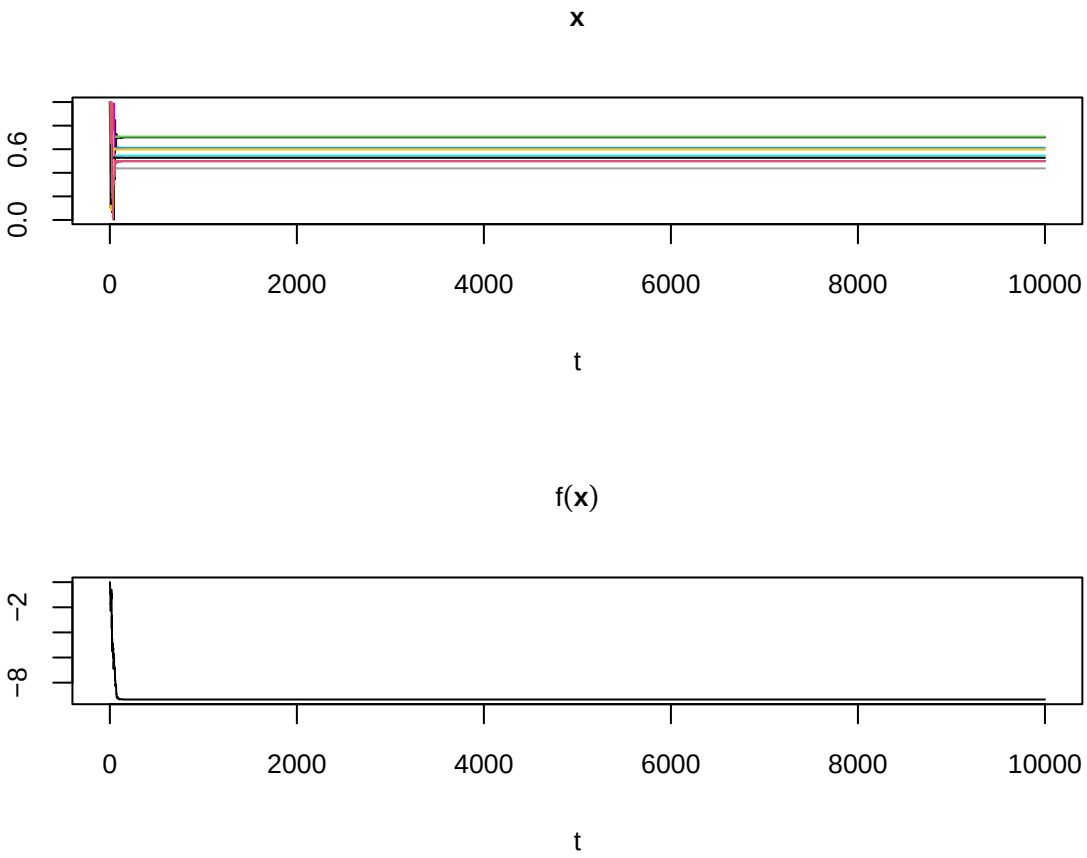

Figure 6: Visualization of the trace

```
      return(list(xt = xt, vt = vt))
    }else{
      return(list(xt = x, vt = vt))
    }
  }

  t = 1
  x.opt.t = x.opt[, t]
  while(t < n.iter){
    print(t)
    for(j in 1:ncol(x)){
       par = update.par(x[, j], v[, j], alpha, gamma)
       x[, j] = par$xt
       v[, j] = par$vt
       if(f(x[,j]) < f(p[,j])){
         p[, j] = x[, j]
         if(f(p[,j]) < f(x.opt.t)){
           x.opt.t = p[, j]
         }
       }
    }
    x.opt = cbind(x.opt, x.opt.t)
    t = t + 1
    obj = c(obj, f(x.opt[,t]))
    x.opt.t = x.opt[, t]
    if(max(abs(obj[t]-obj[t-1])) < 10^(-3)){
      disturb = (matrix(sample(c(-1, 1), s*10, replace = TRUE), 10, s)
                 *matrix(rnorm(s*10, 0.001, 0.0005), 10, s))
      x = x + disturb
      v = v + disturb
      print(TRUE)
    }
  }
  return(list(x = x.opt, obj = obj, time = t))
}
```

```
set.seed(111024513)
opt = pso.sgdm(1000, 1000, alpha = 0.0001, gamma = 0.9)
dump("opt", "appendix_A3.RData")
```

4.3.2 Results

Figure 7 shows the trace for the algorithm. We can see that the algorithm leads to a convergent result. For the final solution for $\boldsymbol{x}$, please see Table 8. Eventually, it is found that the minimized value of $f(\boldsymbol{x})$ is -9.1921.

```
par(mfrow = c(2, 1))
x = opt$x
obj = matrix(opt$obj, 1, length(opt$obj))
matplot(0:(ncol(x)-1), t(x), col =1:nrow(x),
        xlab = "t", type = "l", ylab = "", lty = 1,
        main = expression(bold(x)),
        cex.main = 0.8, cex.lab = 0.8, cex.axis = 0.8)
```

```r
matplot(0:(ncol(obj)-1), t(obj), col =1,
        xlab = "t", type = "l", ylab = "", lty = 1,
        main = expression(f(bold(x))),
        cex.main = 0.8, cex.lab = 0.8, cex.axis = 0.8)
```

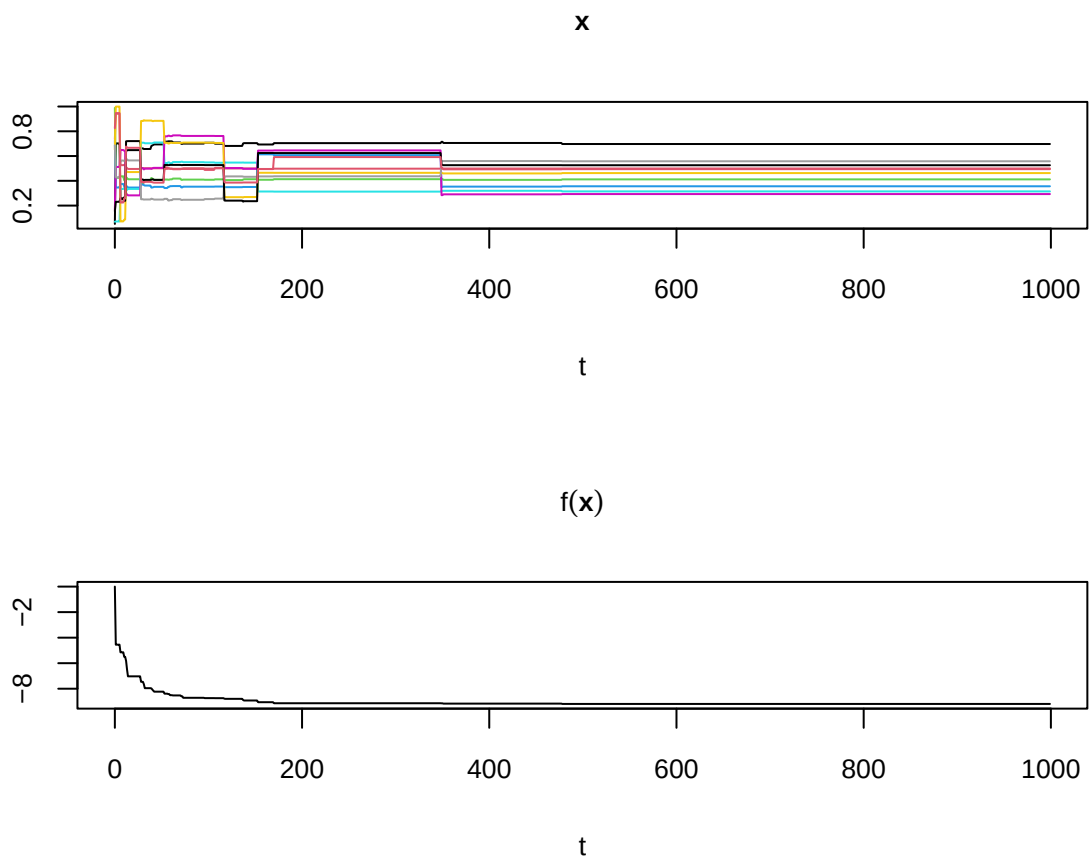

Figure 7: Visualization of the trace

Table 8: Final solution for $\boldsymbol{x}$

x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}
0.6973	0.4942	0.4117	0.3558	0.3145	0.293	0.4625	0.5579	0.5258	0.4994